

CHAPTER 2: LITERATURE REVIEW

2.1 *Introduction*

This will be a review based on all the studies collected on mobile application vulnerabilities. Mobile applications have been the norm and most importantly, the go-to endpoint for activities in your everyday life such as healthcare, digital banking and e-governance. The vast growth of mobile application reliance has been accompanied by the growth of attack surfaces. All 40 of the studies were analyzed to compare vulnerabilities, ways to mitigate, and ways to detect.

2.1.1 *Background of the Study*

The studies address the vast growth of mobile application use in important operations. This is because it has become one of the most convenient ways for them to operate. Operations such as healthcare management have transitioned most of the processes into mobile platforms to add efficiency and accessibility. Though the transition also means an opportunity for attack surfaces to arise.

2.1.2 *Scope of Reviewed Literature*

Four themes have been deduced after thorough evaluation which are Automated Vulnerability Assessment and Intelligent Threat Detection, Sector-Specific Security Postures and Regulatory Compliance, Cross-Platform Ecosystems, and Mobile Cloud Computing (MCC) Architectural Security.

2.2 *Thematic Review of Mobile Application Security*

2.2.1 Automated Vulnerability Assessment and Intelligent Threat Detection

Automated vulnerability assessment is the combination of Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST) and Machine Learning is the trifecta for auditing mobile codebases and being able to detect emerging threats. Static scanning tools are the foundation for code auditing (Chandra et al., 2023; Kusreynada & Barkah, 2024). The recent rise of tools such as SaveVision (Mittal et al., 2026) and ScanDroidX (Anthony et al., 2026) help get analysis on cryptographic analysis and MITRE ATT&CK mapping, providing more coverage. The gaining occurrence in researchers reaching for integrated SAST/DAST pipelines that use Frida and Appium (Swamy et al., 2025; Nagarajan et al., 2025) when they want to validate dynamic behaviors has also been a new topic. ML algorithms such as Multilayer Perceptron (MLP) (Kushwaha et al., 2023), Random Forest (Singh et al., 2025), and CMC permission reduction (Jain et al., 2023) are used to find zero-day malware. MLSTC (Kumari & Sharma, 2023) were proposed to check network packets. It acted as the door before letting the packets in so that the operating system can execute them. A problem with static analyzers is the wide range in performance, ranging from 22% to 67% (Zhu et al., 2024). This led to the rise of new more effective methods such as hybrid pipelines (Swamy et al., 2025; Nagarajan et al., 2025) and dynamic instrumentation (Kusreynada & Barkah, 2024) which have successfully found runtime flaws such as SSL pinning bypasses. ML-based classifiers have great detection accuracy with models that have reached 99.40% accuracy, the problem is that the accuracy is based on clean datasets as opposed to real-world malware (Zhu et al., 2024; Cinar & Kara, 2023). The MLSTC chips have remained as prototypes with no signs of official deployment. The pattern shows that research in mobile defenses is neglected with the research gap of the need of dynamic testing pipelines that are built into the CI/CD workflows.

2.2.2 Sector-Specific Security Postures and Regulatory Compliance

Multiple operations have relied on mobile applications, even with their worrying security posture. Several healthcare applications were included in this study. Indonesia's PeduliLindungi and Mobile JKN have showed frequent OWASP vulnerabilities ranging from deprecated cryptographic hashes to exposed API keys (Priambodo et al., 2022). Cao et al. (2025) did a huge number by reviewing 213 healthcare applications which showed that only 75.9% of the apps had confidentiality mechanisms, 37.4% of the apps were lacking in data integrity and 62.7% of the apps had poor system availability. Abdullah et al. (2024) provided a solution for secure Electronic Health Records (EHR) by having a lightweight AES-256 and Elliptic Curve Cryptography (ECC) hybrid model. Zhu and Chen (2022) proposed multi-level access control for QR in the medical department. Financial and healthcare prioritize custom SDK controls (Kothari et al., 2024), RASP (Blanco-Muñoz et al., 2024) and hybrid AES-ECC encryption (Abdullah et al., 2024). Although they still lack in preventing permission manipulation and have weak authorization logic (Mykhaylova et al., 2023; Al-Hakimi et al., 2025). The studies go between post-hoc dynamic audits (Falade & Ogundele, 2023; Al-Delayel, 2022) and proactive security frameworks (Al-Sherideh et al., 2024; Chowdhury et al., 2025). The studies have shown the systematic vulnerabilities throughout commercial applications but do suffer from basing the results on a small sample size (Al-Hakimi et al., 2025; Priambodo et al., 2022). Also, the preventative measures are mostly reactive instead of having them implemented during development (Chandra et al., 2023; Abdullah & Zeebaree, 2021). There is common occurrence of basic cryptographic and storage errors that would fail the regulatory guideline. This makes them lack security, making a gap for Secure Software Development Life Cycle (SSDLC) frameworks.

2.2.3 Cross-Platform Ecosystems, Network Transport, and User Privacy

There has been rapid growth in cross-platform development frameworks. Although they do come with insecure network transport channels that give way to new attack vectors. Cross. Fedynyshyn et al. (2024) analysed 6165 APKs which showed that Xamarin apps had the most binary flaws and React Native had the most code-level vulnerabilities. Rambhia et al. (2023) had a counter towards the claim that Flutter’s compilation blocks tampering by proving that Flutter binaries were vulnerable to dynamic memory manipulation. The only way to counter this is by explicitly implementing the security measures beforehand. The binary analysis has shown that Xamarin applications have a high vulnerability rate, whereas React Native apps have high source-code permissions (Fedynyshyn et al., 2024). PrivProt shows the minimal latency (Cevallos-Salas et al., 2025), but audits Sakuraba et al. (2025) say that commercial apps fail to provide basic encryption. The research gap is the amount of disconnect between the designated privacy policies and the actual runtime network behaviour. It shows the need for automated verification gateways that would then validate the network traffic.

2.2.4 Mobile Cloud Computing (MCC) and Architectural Security

Mobile Cloud Computing would offload computational tasks to the backend infrastructures which would then solve the hardware limitations issue but also open the gate to complex ecosystem vulnerabilities (Sheikh et al., 2023; Alqarqaz & Younes, 2022). Tang et al. (2022) created vTrust which is a remote Trusted Execution Environment (TEE). Within a remote cloud virtual machine, it processes the application, uses a local hypervisor that encrypts device screen frames and touch inputs (AES-128-GCM), and blinds the compromised OS (Tang et al., 2022). Tandri et al. (2022) showed how efficient the cloud offloading was by running a resource-heavy sign-language translation app using Google App Engine (PaaS). It reduces the clients’ CPU utilization to under 2%, yet the latency was just 6.07 ms. vTrust (Tang et al., 2022) and PaaS API designs (Tandri et al., 2022) have solved the resource limitations. On the other end, multi-tenant app-in-app ecosystems (Zhang et al., 2024) have the opposite results by proving that sharing resources between host apps give severe cross-party impersonation risks. Cloud offloading solutions have shown great proof of good performance by reducing the client CPU utilization to under 2% with low latency (Tandri et al., 2022). Although a problem with MCC literature is that it’s tested in controlled environments (Tang et al., 2022; Tandri et al., 2022) or theoretical taxonomies (Chimuco et al., 2023; Sheikh et al., 2023; Alqarqaz & Younes, 2022) without actual proof of it reducing latency in 6G network conditions. The findings show that mobile applications are evolving. Traditional single-device security models have become a thing of the past, meaning the need for multi-party Zero Trust authorization models in the future.

2.3 Literature Review

2.3.1 Literature Review Analysis Table

No.	Name	Year	Author(s)	Objective / Problem	Dataset / Data Source	Method / Approach	Functionality / Process	Tools / Technologies	Evaluation Metrics	Key Findings / Results	Weaknesses / Limitations	Research Gap / Relevance
1	PrivProt: A Privacy-Aware Application Layer Protocol for Mobile Environments	2026	Sangeeta Mittal Shreyash Gajbhiye Sneha Arora	proposes an AI-based static analysis tool for detecting cryptographic vulnerabilities in mobile applications before deployment or installation	Analyzed 5 categories Social, Media, Shopping, Gaming Entertainment and Productivity Use APK and IPA files	Use static analysis with keyword-based AI/NLP model to identify cryptographic weakness	Check APK /IPA files analyzes code and configuration files, detects cryptographic weakness and creates a security report	Apktool, Androguard, Smaili, AndroidManifest.xml, info.plist	Number or Vulnerabilities, risk category and CVSS score	All 48 apps had atleast 1 cryptographic vulnerability	Small dataset,a mainly static analysis and no standard AI performance metrics	Futeure studies can use more apps, have dynamic analysis, and measure the accuracy

2	Mobile Application Security in the Health Care and Finance Sector with Static Analysis (Tools: MobSF)	2025	Majeti Srinadh Swamy; P Pramod Kumar; Sunil Kumar Thota; Rathan Kumar Chenoori	To create a cross-platform automated testing framework that evaluates usability and security of mobile app together	5 android and 5 iOS apps from finance, healthcare, education and entertainment, collects interaction logs	cross-platform automation, machine learning for usability analysis and Static Application Security Testing (SAST) + Dynamic Application Security Testing (DAST) for security testing	The system simulates user actions, collects interaction logs, analyzed usability, scans for security vulnerabilities and combined into a report/dashboard	Appium or Detox, MobSF, Custom static/dynamic analyzers, ML models, Android and iOS Platforms	Task completion rate, Task time, user satisfaction, vulnerabilities detected, severity score, false positive, execution time and devices tested in parallel	The proposed framework achieved 94% task completion, 18.2s task time, 4.6/5 satisfaction, detected 16 vulnerabilities, had 8% false positives and took 40 minutes while testing 5 devices in parallel	ML Models need retraining for different app domains and dynamic analysis require high computing recourse for large application	More testing is needed across larger and more diverse mobile apps and devices. Better Cross-platform usability-security metrics and scalable automation.
3	A Comprehensive Study on Static Application Security Testing (SAST) Tools for Android	2024	Jingyun Zhu, Kaixuan Li, Sen Chen, Lingling Fan, Junjie Wang, Xiaofei Xie	To compare Android Static Application Security Testing (SAST) tools fairly. The paper proposes VulsTotal, a unified platform for evaluating different SAST tools	11 Android SAST tools were evaluated using synthetic benchmarks (GHERA, MSTG, PIVAA) and a new real-world CVE benchmark containing 250 CVEs, 229 APKs, 262 vulnerability instances, and 34 vulnerability types	The researchers unified 67 vulnerability types, normalized tool reports, created a CVE-based benchmark, and compared the tools using the VulsTotal platform	VulsTotal takes Android APKs, runs multiple SAST tools, converts their different reports into a common format, maps findings to a unified vulnerability taxonomy, and compares the tools	MobSF, QARK, AndroBugs, APKHunt, SUPER, JAADAS, DroidStatx, Marvin, Trueseeing, AUSERA, SPECK, Python, Java, Go, Rust, Scala, Soot, FlowDroid, and Androguard	Vulnerability type coverage, type consistency, detection effectiveness, recall, and scanning time/performance	APKHunt had the highest coverage at 67%, AUSERA reached 60%, while JAADAS had only 22%. The study concludes that using multiple SAST tools together is better for comprehensive Android vulnerability detection	The real-world benchmark mainly relies on CVE data, only general-purpose SAST tools were evaluated, and tools were tested using their default configurations	Future work should improve SAST detection methods, create broader real-world benchmarks, and combine multiple analysis techniques/tools
4	Android Apps Vulnerability Detection with Static and Dynamic Analysis Approach using MOBSF	2023	Rudy Chandra; Togu Novriansyah Turnip; Eka Stephani Sinambela; Goklas Henry Agus Panjaitan; Vivi Nessa Tampubolon; Agnes Mebyolla Turnip	To analyze and compare the security of healthcare and finance mobile application with static analysis	5 Healthcare and 5 Finance android apps APK files from ApkPure	Static Application Security Testing (SAST) using MobSF with vulnerabilities scored using CVSS v3.1	APK files are uploaded to MobSF, which scans the application code, identifies scored using CVSS3.1	MobSF, CVSS calculator, Git, Python, OpenSSL, wkhtmltopdf, CWE, OWASP, MASVS and MSTG	CVSS vulnerability score and vulnerability categories such as Storage, cryptography, network, platform and resilience	The main vulnerabilities were related to storage, cryptograpy, network, platform and resilience	The study mainly uses static analysis and cannot fully detect runtime issues. Only 10 android applications were tested	Future research should combine static and dynamic analysis
5	SaveVision: AI-Based Tool for Detecting Cryptographic Vulnerabilities in Mobile Applications	2022	Yutao Tang; Zhengrui Qin; Zhiqiang Lin; Yue Li; Shanhe Yi; Fengyuan Xu	To protect security-sensitive mobile apps and their data even when the mobile operating system is untrusted or compromised. The paper proposes vTrust, a remote trusted execution environment	The prototype was tested using 7 typical mobile apps, an Arndale development board, and a remote Android-x86 server	Virtualization-based Trusted Execution Environment (TEE) with remote app execution and encrypted communication between the trusted mobile hypervisor and remote server	Sensitive apps run inside a remote VM. Mobile inputs such as touch and sensor data are encrypted by the Hyper Stub, transferred through the untrusted OS, and decrypted on the server. App outputs are encrypted and returned to the mobile device	Virtualization, KVM, QEMU, VMware, Android-x86, AES-128-GCM, SHA-256, LZ4 compression, VPN, hypervisor, and virtual machines	Encryption/decryption overhead, compression ratio, frame throughput (FPS), input latency, output latency, and security against attacks	Encryption/decryption overhead, compression ratio, frame throughput (FPS), input latency, output latency, and security against attacks.	vTrust depends on a trusted hypervisor and remote VM, cannot defend against attacks such as DoS, has lower FPS than local execution, and does not currently support audio output	Future research can improve network performance, multimedia support, attack resistance, and deployment on modern mobile devices, while reducing latency and dependence on trusted infrastructure
6	Plaintext in the Wild: Investigating Secure Connection Label Accuracy for Android Apps	2024	Trishla Shah; Raghav Sampangi; Angela Siegel	To evaluate the security risks of sensitive user inputs in Android apps using the Hierarchical Weighted Risk	About 210 Android apps from Google Play Store, covering 7 categories such as social networking, finance, health, e-commerce,	HWRSR risk scoring combined with static analysis and manual validation.	The framework extracts sensitive inputs, checks whether data is stored, masked, obfuscated, or shared, assigns weights to each test, applies sensitivity	Amandroid, APKTool, static analysis, HWRSR, Java API/method analysis, and Google Play Store data	Risk score from 0–100, alignment with manual evaluation, higher/lower model scores, and category-based comparison	Out of about 210 apps, the model scores matched manual evaluation in 175 cases. It produced higher scores in 19 cases and lower scores in 7 cases	The model may underestimate risk when several vulnerabilities exist, may fail to extract sensitive terms correctly, and may not recognize custom	Future work should improve risk aggregation and build a database of custom encryption/obfuscation methods to make the model more accurate

				Scoring Model (HWRSM).	education, food delivery, and productivity.		factors, and calculates an overall risk score.				encryption or obfuscation methods	
7	Risk-Aware Mobile App Security Testing: Safeguarding Sensitive User Inputs	2025	David Cevallos-Salas; José Estrada-Jiménez; Danny S. Guamán	To protect personal data used by mobile apps from threats that can intercept data before HTTPS/TLS encryption. The paper proposes PrivProt, a privacy-aware application-layer protocol	Used 50 authentication trials and 50 personal-data transmission trials on both a physical and virtual Android device	Application-layer encryption, ECDHE key exchange, AES-256 encryption, OAuth 2.0 tokens with privacy claims, and REST-compatible communication	creates a secure application-layer session, then authenticates the user, generates an OAuth 2.0 token, checks whether the destination server is authorized, and finally sends encrypted personal data	ECDHE, AES-256, OAuth 2.0, JSON, REST, TCP, P-521 elliptic curve, Android 13, Xiaomi Redmi Note 13, and Windows 11 server	Round-Trip Time (RTT), mean, median, standard deviation, and p-value using the Wilcoxon rank-sum test	PrivProt was faster than HTTP. On the physical device, mean RTT was 23.08 ms vs 58.82 ms for control messages and 29.44 ms vs 65.16 ms for data. On the virtual device, it achieved 10.12 ms vs 23.58 ms and 11.28 ms vs 24.98 ms	PrivProt does not authenticate the remote server by itself and is intended to complement, not replace, TLS/SS. It also does not yet provide broader properties such as availability and non-repudiation	Future work should add integrity, availability, remote-server authentication, non-repudiation, and new cryptographic methods, while maintaining compatibility with mobile OSs and REST
8	Machine Learning for Mobile Defense: Detecting SMS Malware and Riskware on Android	2024	Sabrina Uhti Kusreynada; Azhari Shouni Barkah	To identify security vulnerabilities in the Mobile JKN Android healthcare application, which stores sensitive personal and medical information.	One Android application: Mobile JKN APK from BPJS Kesehatan	Combination of static analysis and dynamic analysis using the Mobile Security Framework (MobSF)	Static analysis checks permissions, certificates, AndroidManifest.xml, source code, and malware domains. Dynamic analysis runs the app and tests API monitoring, SSL pinning bypass, root detection bypass, and debugger checker bypass	MobSF, Android APK, Android emulator, SSL/TLS Pinning, SQLite, AndroidManifest.xml, SHA-1, CBC encryption, and WebView analysis	vulnerability findings and severity results from static and dynamic testing	Static analysis found 5 dangerous permissions, weak cryptography, hardcoded secrets, insecure WebView, and vulnerabilities to Janus, SQL Injection, and padding oracle attacks. Dynamic analysis showed SSL pinning worked properly, but root detection and debugger detection were weak	The study analyzes only one Android healthcare app and mainly relies on MobSF	Future research can test more Android apps, combine MobSF with additional penetration-testing tools, and compare static and dynamic detection results using measurable metrics such as detection rate and false positives
9	Cross-Platform Automated Testing Framework for Mobile App Usability and Security	2025	Yusei Sakuraba; Hiroki Inayoshi; Shoichi Saito; Akito Monden	To check whether Android apps' "secure connection" labels in Google Play accurately match their real network behavior	12,426 top-ranked Android apps from all 35 Google Play Store categories, collected in 2024 and 2025	Dynamic analysis of app network traffic, followed by searching for sensitive data sent through plaintext HTTP and comparing it with the app's declared security label	Apps are collected and executed, network traffic is captured, sensitive information is searched in HTTP traffic, and the results are compared with Google Play secure connection labels	Python, ADB, Google Pixel 8a, Squid proxy, tcpdump, mitmdump/mitmproxy, HTTP/HTTPS traffic analysis, and Android UI automation	Number of apps using HTTP/HTTPS, number of plaintext sensitive-data leaks, label consistency, app install counts, and types of leaked data	65 apps transmitted sensitive data without encryption. 46 apps falsely claimed their data was encrypted, while 21 apps correctly disclosed insecure transmission	Sensitive data search values were manually selected and may miss some leaks. App interaction was limited and traffic collection methods differed between datasets	Future work should develop automatic tools that verify or generate secure connection labels, using stronger static and dynamic analysis to improve label accuracy
10	VTrust: Remotely Executing Mobile Apps Transparently with Local Untrusted OS	2025	Anmol Rattan Singh; Gurjinder Singh; SriPELL Jagadish; Amaar Haamid Shnaeen	To detect SMS malware and riskware on Android devices using machine learning and improve user privacy and security	CICMalDroid2020 dataset, containing about 17,341 Android applications including adware, banking malware, SMS malware, riskware, and benign apps	Machine learning classification using Logistic Regression, Random Forest, and Naive Bayes. The study also discusses static and dynamic Android malware analysis	APK files are collected, analyzed by malware detection software, classified as malicious or safe, and malicious data/apps are blocked or removed	Machine Learning, Android APK analysis, Logistic Regression, Random Forest, Naive Bayes, and CICMalDroid2020	Accuracy, Precision, Recall, and F1-score	Random Forest performed best, with the highest accuracy, recall, and F1-score	The study relies mainly on one existing dataset, and malware continues to change over time	Future research should improve and update ML models to detect new and evolving Android malware, including more advanced and zero-day threats
11	The Current State and Future of Mobile Security in the Light of the Recent Mobile Security Threat Reports	2023	Ahmet Cevahir Cinar; Turkan Beyza Kara	To evaluate the current state of mobile security based on mobile security threat reports	Kaspersky Security Network report	A systematic literature review and qualitative analysis	Classifies attack types and puts malware detection techniques into two categories, signature-based detection and machine learning detection	Signature-based tools, machine learning algorithm, data encryption, and data summarization	Evaluation is done qualitatively	Machine learning methods are better than signature-based methods	Doesn't introduce a new defense mechanism and relies on third party threat reports	Current mobile malware defense mechanisms are struggling to keep up with threats that continue to adapt at a fast pace
12	Mobile Application as a Critical	2023	O. Mykhaylova;	To evaluate the security of a mobile	The mobile application of the	Static and Dynamic analysis	Automated penetration testing, malware analysis,	Mobile Security Framework (MobSF)	Qualitative and vulnerability focused. Scores are given	Lack of usage confirmation that would allow attackers	Research based on one singular mobile application, therefore	Lack of secure development lifestyle within government

	Infrastructure Cyberattack Surface		T. Fedynyshyn; A. Datsiuk; B. Fihol; Hennadii Hulak	application used by Ukraine's postal operators	Ukrainian postal service operator	techniques, basing assessment on cybersecurity guidelines and frameworks	and security assessment, checking which attack were exploitable within an emulated environment		whether security flaws have been found	to access sensitive information	narrowing the scope. Vulnerabilities would be implied to only that mobile application	applications. Adaption of DevSecOps methodology during the development process is highly encouraged
13	Development of a Secure Model for Mobile Government Applications in Jordan	2024	Ala'a Saeb Al-Sherideh; Roesnita Ismail; Mohammad Rasmi Al-Mousa; Khaled Al-Qawasmi; Ala'a Al-Shaikh; Hebatullah Awwad; Khaled Maabreh; Mohammad Alauthman	To develop and validate a secure privacy model for mobile applications in Jordan	Interviews with 8 Jordanian e-government experts; Questionnaire to 150 Jordanian citizens	Mixed-methods approach. Qualitative through the experts and quantitative through the citizens	Identifying vulnerabilities and privacy measures through the experts, implementing it into a prototype application, having end-users interact with the prototype	Android programming, SMS-based One-Time Passwords, data encryption, login tracking mechanisms	User acceptance metrics through the questionnaire given to the 150 citizens	Experts verified that effective privacy features are important for sensitive M-G services, implementation of security measures improved the security of the app, citizens consensus is that it's easy to use	Geographic scope, only given to citizens in Jordan, demographic scope, 150 citizens is a small sample size	Mobile government applications have a lack of security measures while also providing practicality. Future government applications should emphasize practical security features
14	The Dark Forest: Understanding Security Risks of Cross-Party Delegated Resources in Mobile App-In-App Ecosystems	2024	Zhibo Zhang; Lei Zhang; Guangliang Yang; Yanjun Chen; Jiahao Xu; Min Yang	To evaluate the security regulations and weaknesses of Cross-Party Delegated Resources in mobile application ecosystems	9 mobile app-in-app ecosystems	A systematic empirical analysis of real-world app-in-app ecosystems	Map out resources from host apps to sub-apps to identify security regulations, try to exploit the ecosystem using three attacks, vulnerabilities were shown to the developers	Mobile App-in App Ecosystems, Cross-Party Delegated Resources	Metrics done by evaluating the vulnerabilities of the 9 ecosystems, success rates of the 3 attacks	Violation of 11 common security regulations across the 9 ecosystems	Focusing on 9 ecosystems would create a variety in results. Attack vendors would also need to keep up as the platform has rapid update cycles	The research defines the regulation for Cross-Party Delegated Resources
15	Secure Cloud-Based Mobile Apps: Attack Taxonomy, Requirements, Mechanisms, Tests and Automation	2023	Francisco T. Chimuco; João B. F. Sequeiros; Carolina Galvão Lopes	To present a proper framework for attack taxonomy, security mechanisms and security test specifications for cloud and mobile ecosystem	Systematic review of cybersecurity literature, vulnerability databases and software testing tool reviews	Systematic and qualitative approach. Deconstructing cloud-mobile ecosystems, assessing the vulnerabilities and categorizing them into a novel	Maps out how mobile application and cloud infrastructures communicate, assess known vulnerabilities, categorize attacks, define security requirements, outline how the security testing tool should be made to test the entire ecosystem	Mobile platforms, Cloud Computing Service Models, Automated Security Testing Tools	Qualitative by having the taxonomy covers known threats and testing specifications	Creates Mobile and Cloud combined attack taxonomy. Provides detailed specifications on how security testing tools should be built	It shows how the testing tools should be built but would serve as an implementation struggle	Closing the gap between cloud and mobile app security would ensure that it isn't vulnerable to attacks from front to back
16	Vulnerability Analysis of Digital Banks' Mobile Applications	2023	Polra Victor Falade; Grace Bunmi Ogundele	To assess the security of mobile applications of UK-based digital banks	6 digital bank applications	Automated vulnerability assessment by Androbugs security scanner	APK files for the six applications were ran through Androbugs security scanner to assess vulnerabilities	Androbugs security scanner	Identification of security vulnerabilities that were found in the APK files	All six of the digital banking applications had vulnerabilities	Small sample size of six applications. Relying on the security scanner could produce false positives	Shows that major banks have vulnerabilities in their digital banking app, proving a need for security testing throughout the lifecycle
17	Security Analysis of Mobile Banking Application in Qatar	2022	Shaymaa Abdulla Al-Delayel	To evaluate the security posture of mobile banking applications in Qatar	2 mobile banking applications	Vulnerability assessment by running the mobile banking applications through a combination of two mobile security frameworks	Putting the APK files through the automated static and dynamic security testing, the tools identified security gaps, vulnerabilities and flaws in the data storage	Android applications, NowSecure Lab Automated tool, Quick Android Review Kit	Identification and categorization of the security vulnerabilities, putting the weaknesses against standardized mobile security practices	The mobile banking applications lacked security practices and had major vulnerabilities. Issues contained in cryptographic implementation and data handling	Small sample size of two mobile banking applications	There was a lack of security evaluation in Qatar, therefore bringing light into prioritizing security by design to maintain end-user confidence in its ecosystem
18	A Proposed Taxonomy of Cybersecurity	2022	Muhammad Noman Khan	To present a taxonomy for mobile	Comprehensive literature review	Systematic literature review and qualitative	Analyzes mobile applications security risks and categorizes them into a	Mobile application	The comprehensiveness of the taxonomy in recognizing all types	Mobile malware is more prominent towards mobile banking applications.	It is theoretical and relies on existing literature	Shows the lack of structured categorization for mobile threats. A

	Risk in Mobile Applications			application security risks		analysis. Identifying common security threats, categorizing them according to attack vendor	taxonomy. Breaking down the mobile ecosystem into components and then mapping threats to these components		of cyberattacks on mobile applications	Android is more valuable as it is open source		continuously updating taxonomy would help experts to accurately identify threats
19	Post Hoc Security and Privacy Concerns in Mobile Apps: The Moderating Roles of Mobile Apps' Features and Providers	2024	Hamid Reza Nikkhah; Varun Grover; Rajiv Sabherwal	To investigate how user security concerns impact their use of Mobile Cloud Computing apps	Survey of 694 MCC app users	Quantitative survey using Privacy Calculus Theory	Separate the privacy concerns and security concerns and have the 694 users give their perceptions. Then analyze the data using Stata 15.0 to measure the effects of the variables	Mobile Cloud Computing (MCC), ISO 27018, Data Assurance Mechanisms, Stata 15.0	The perception of privacy concerns, security concerns, value of data transfer, effectiveness of security interventions	The perception of the security and privacy is negative. Trust only accommodates privacy but not security	The answers of the survey are based on hypothetical scenarios, not actual experiences	By separating security and privacy concerns, it showed that while privacy concerns have reduced, security concerns are still very present
20	Mobile Cloud Computing: A Survey on Current Security Trends and Future Directions	2023	Bisma Sheikh; Ayesha Butt; Javeria Hanif	To provide a comprehensive literature review on Mobile Cloud Computing (MCC)	Academic publications and security schemes about MCC	Narrative literature review and survey	Evaluate the MCC landscape and define the core architecture, comparing the benefits of offloading data to the cloud and bottlenecks, review cryptographic and authentication schemes in past literature and see how the future of MCC will be	Cellular communication networks, SSL, digital certificates, Modular Encryption Standards, AR, VR, MR	Qualitative metrics about data security efficacy, network latency, computational concentration, and bandwidth management	MCC mitigates hardware limitations as it has dynamic provisioning and network availability	Depends on existing papers instead of testing it	It connects cryptographic solutions with the future of 6G that would need to support AR and VR
21	Android Mobile Applications Vulnerabilities and Prevention Methods: A Review	2021	Hilmi Abdullah; Subhi R. M. Zeebaree	To investigate common mobile application vulnerabilities and their respective prevention methods focusing on the android platform	Security reports, existing literature and OWASP mobile risk benchmarks	Systematic review and literature analysis of mobile security threats, exploits and defense mechanisms	Reviewing how attackers exploit vulnerable mobile apps to execute malicious code and access private data alongside OWASP top mobile risks	OWASP framework guidelines, android security architecture	Qualitative analysis of vulnerability types, frequency of attacks and mitigation effectiveness	Highlighted that mobile app developers must prioritize security and data privacy during development to counter rising malicious code execution and data theft	Provides a broad review without empirical runtime experimentation or a novel automated detection tool.	Identifies a need for proactive, automated vulnerability identification mechanisms during early app design and development stages
22	Vulnerabilities in Government Mobile Applications: Evaluating the Security of Mobile Apps in Indonesia and Malaysia	2025	Asmaa Mahfoud Al-Hakimi; Nyoman Gede Arya Astawa; Wayan Budi Sentana; Made Ari Dwi Suta Atmaja	Evaluate and analyze security vulnerabilities present in official government mobile applications across Indonesia and Malaysia	Selected public sector and government mobile applications deployed in Indonesia and Malaysia	Security evaluation framework utilizing vulnerability assessment, static/dynamic code review, and penetration testing principles	Scanning mobile app packages, analyzing client-server communication, checking data storage permissions, and identifying implementation flaws	Mobile application security testing tools, automated scanners, and penetration testing frameworks	Vulnerability severity ratings for example; high, medium, low; Frequency of insecure configurations, and OWASP-aligned risk categories	Highlighted critical security flaws such as insecure data storage, weak transport layer protection, and permission mismanagement across several evaluated government apps	Scope was restricted to specific government sectors in two countries; runtime/dynamic behavioral monitoring under live deployment was limited	Identifies the need for standardized, automated security hardening guidelines for cross-border public sector mobile applications
23	Perception of Information Security in Mobile Applications	2022	Diana Encalada; Milenny Soto-Alvarado; Viviana Chamba-González	Analyze mobile application security and privacy issues from both a theoretical and empirical perspective, focusing on user perception	Survey data gathered from mobile application users across different generations	Mixed-methods approach incorporating a literature review and an empirical survey to evaluate user perception and awareness	Surveying user habits, trust levels regarding permissions, and awareness of data sharing risks within mobile application environments	Survey instruments, statistical and descriptive data analysis frameworks	User awareness levels, perceived insecurity percentages, generational trust variance, and data-sharing tolerance	Revealed poor user knowledge regarding shared data security alongside high general insecurity about how apps handle personal information. Notably, younger generations showed higher trust and permitted greater app access compared to adult populations	Relies on self-reported survey data rather than technical runtime analysis or behavioral log tracking of actual mobile app usage	Highlights the human factor and user-awareness gap in mobile security, indicating that technical defenses must account for end-user misconfigurations and over-permissive consent habits
24	Comprehensive Research on Mobile Application	2025	Nagarajan MK; Rayavarapu Lakshmi	Address the limitations of individual security	Android application packages (APKs) evaluated for	Hybrid framework combining static analysis,	Decompiling APK code for low-level structural review, performing dynamic	MobSF for static analysis, Frida for dynamic instrumentation, VirusTotal for malware detection, and Apktool/Quark for reverse engineering and API call analysis	Detection accuracy, false-positive reduction rate, and effectiveness against	The integrated hybrid framework significantly improves detection accuracy,	Implementation relies heavily on signature and tool coordination; requires further	Suggests incorporating machine learning enhancements to optimize zero-day

	Security Assessment		Narasimha Dinesh; Ratna Selvan S; Mothadaka Nagamaiah; Kolakani Venkata Prasanna Anjaneyulu	approaches by proposing a comprehensive hybrid framework to detect both known and unknown mobile malware and vulnerabilities	runtime and static behaviors	dynamic analysis, reverse engineering, and malware analysis techniques	instrumentation, scanning for behavioral patterns, and cross-referencing malware signatures		known and unknown threats	minimizes false positives, and effectively exposes hidden security defects and malicious payloads	automation enhancements	threat detection within automated mobile security assessment pipelines
25	Exploration of Application Security for Medical Electronic Health Card	2022	Jiang Zhu; Zhenyu Chen	Address information security vulnerabilities and compliance requirements in medical electronic health card systems, ensuring reliable protection for online-offline medical services	Medical electronic health card applications, QR code identification architectures, and online-offline medical service networks	Security architecture design and risk evaluation structured around multi-level information security protection requirements	Implementing security authentication, secure data transmission protocols, and unified national identity credentials for Internet medical services using technologies like QR codes	Secure cryptographic authentication protocols, electronic health card system platforms, and online-offline integration mechanisms.	Information security level protection compliance, authentication strength, and transmission security performance	Demonstrated that establishing unified security requirements across online and offline channels significantly minimizes unauthorized access risks and protects patient digital medical assets	Primarily focuses on structural framework compliance and system-level architecture without extensive empirical performance testing under heavy live transaction volume	Highlights the need for continuous runtime security monitoring and lightweight authentication for resource-constrained smart health card applications
26	Security Model with a Holistic Approach for Mobile Applications of Financial Entities in Colombia	2024	Carlos Mauricio Blanco Muñoz; Diego Alejandro Clavijo Ladino; Jhonnyer Danilo Nuñez Castillo	Address the growing landscape of cyber threats targeting mobile banking channels by establishing a comprehensive security model tailored for financial institutions in Colombia	Financial sector cybersecurity metrics, mobile banking threat reports, and regulatory frameworks governing Colombian financial institutions	Holistic security framework design integrating lifecycle app security, threat modeling, and defensive controls across client, network, and server tiers	Implementing end-to-end protection processes, including secure authentication, session management, data encryption in transit and at rest, and continuous runtime application protection (RASP).	OWASP Mobile Top 10 standards, mobile vulnerability assessment tools, cryptographic encryption suites, and secure API gateways	Compliance coverage with financial regulations, threat mitigation success rate, vulnerability reduction across mobile app components, and implementation feasibility	Demonstrated that a holistic architecture reduces critical vulnerabilities such as insecure data storage and weak transport layer controls significantly better than isolated patch methods.	Focused primarily on the Colombian regulatory ecosystem and conceptual framework validation without large-scale deployment telemetry across every commercial bank type	Identifies a need for automated integration of holistic security models directly into continuous integration/continuous deployment (CI/CD) pipelines for financial mobile software
27	Digital Forensics Analysis of a Financial Mobile Application: Uncovering Security and Privacy Implications	2024	Rajon Bardhan; Raymond Garay-Paravisini; Gokila Dorai; Logan Vanputte	Investigate security and privacy vulnerabilities in financial mobile applications through a rigorous digital forensics framework to uncover residual sensitive data left on devices	Financial mobile application artifacts, cached files, local databases, and application state logs extracted from test mobile environments	Empirical digital forensics investigation utilizing storage acquisition, artifact extraction, and content parsing	Inspecting local application directories, parsing SQLite databases, and recovering residual user credentials, session tokens, and transaction identifiers from device memory and storage	Mobile forensic extraction tools, database viewers, and file system analyzers	Presence and readability of sensitive personal/financial data, log verbosity, encryption strength of local storage, and forensic artifact recovery rates	Demonstrated that several financial applications inadvertently leak sensitive user identifiers, transaction artifacts, and session credentials in plaintext or weakly encrypted local storage files, creating notable privacy risks	Focused on specific test financial applications without examining live cross-platform server-side syncing behavior across varying hardware architectures	Highlights the need for stricter local data sanitization, robust application-level encryption for cached files, and automated runtime forensic auditing during mobile app development
28	Blueprint: A Cyber Crime, Personal Security, and Police Assistant Mobile Application Featuring Fraud Investigation and Crime Reporting Features	2025	Mahfuzulhoq Chowdhury; Afroza Akter; Estiak Ahamed Sazid; Md. Zanea Alam; Mohammad Salah Uddin; Kaniz Fatima Tonni	Address limitations in conventional law enforcement tools by designing a comprehensive mobile application that integrates cybercrime detection, fraud reporting, and digital police assistance features	User feedback surveys, app credibility testing data, and standardized crime reporting workflow architectures	Mobile application development and systems design approach combined with user-centered credibility and response evaluation	Providing features such as online general diary (GD) and First Information Report (FIR) filing, fraudulent web link reporting, cyberattack assistance, emergency chat, and lost/stolen item status tracking	Mobile application framework, secure database connectivity, geolocation, and emergency communication modules	User response approval ratings, app credibility assessment scores, and feature completeness compared to legacy tools	Demonstrated positive user reception exceeding 61% approval during credibility testing, successfully bridging communication gaps between citizens and law enforcement officers	Primarily focuses on functional prototyping and user-satisfaction metrics rather than high-load runtime performance or advanced machine learning threat classification	Highlights the integration gap between citizen-facing mobile applications and backend police data systems, serving as a baseline for secure mobile reporting architectures
29	Enhanced Mobile App Security for	2024	Abdullah H. S; Othman O. Khalifa;	Protect sensitive patient healthcare data within mobile	Healthcare mobile application data structures,	Hybrid cryptographic integration framework	Embedding AES-ECC core security protocols into mobile applications to	AES-256 encryption suites, ECC cryptographic libraries, and mobile simulation performance tools measuring CPU time and power consumption	Encryption/decryption execution speed, CPU time overhead, power	Demonstrated that integrating AES with lightweight ECC keys provides robust data	Focused primarily on cryptographic algorithm performance and simulated app	Identifies the need to optimize hybrid cryptographic pipelines for real-time mobile

	Healthcare Applications		Aisha H. A. Hashim	applications against unauthorized access and security breaches while maintaining optimal device performance	cryptographic performance metrics, and security simulation testbeds	combining Advanced Encryption Standard (AES) for data encryption and Elliptic Curve Cryptography (ECC) for efficient key management	safeguard local and transmitted electronic health records (EHR), enforce HIPAA-compliant data confidentiality, and secure key exchanges		consumption, and key security strength	confidentiality and integrity while significantly reducing computational overhead and storage demands on resource-constrained mobile devices	environments without testing against active multi-vector malware or live network attacks	healthcare apps communicating with cloud-backed IoT medical databases
30	ScanDroidX: A Mobile-Native Security Tool with MITRE ATT&CK Mobile Integration for Pre-Deployment Web Application Assessment	2026	Alisha Anthony; Andrew Daniel; Rishabh Tiwari; Dhanashree Huddedar	Address the lack of robust pre-deployment security assessment tools for mobile and web application interfaces by integrating a mobile-native scanner with structured threat frameworks	Pre-deployment web application endpoints, mobile application packages, and simulated vulnerability test environments	Mobile-native security tool design coupled with structural integration of the MITRE ATT&CK Mobile threat taxonomy	Scanning pre-deployment applications for security flaws, mapping detected vulnerabilities directly to adversary tactics and techniques defined in the MITRE ATT&CK Mobile matrix, and generating actionable remediation reports	Mobile-native scanning engines, MITRE ATT&CK Mobile framework, and pre-deployment testing environments	Vulnerability detection coverage, framework mapping accuracy, and pre-deployment risk mitigation efficiency	Demonstrated that integrating standard adversary behavior matrices (MITRE ATT&CK) directly into a mobile-native assessment workflow helps developers identify high-risk attack vectors before production deployment	Primarily focused on pre-deployment staging validation; requires further field testing against highly dynamic runtime application architectures	Bridges the gap between static/dynamic mobile app assessment and real-world adversary behavior modelling, providing a blueprint for proactive pre-release vulnerability management
31	Mobile Health Application Security Assessment Based on OWASP Top 10 Mobile Vulnerabilities	2022	Dimas Febriyan Priambodo; Aldi Cahya Fajar Nugraha; Guntur Satria Ajie, Aulia Rachmawati; Hendy Aulia Rahman; Marcella Risky Avianti	To assess the security of commonly used Indonesian m-Health apps, static analysis was performed to find and rank vulnerabilities that potentially lead to serious data leaks.	We examined the APK files of two popular Indonesian public health apps: Peduli Lindungi and Mobile JKN.	Used a four-phase Static Application Security Testing (SAST) methodology, with results cross-validated against two key analysis tools.	The APK files were extracted, examined for source code flaws, and the results were mapped to established risk frameworks to calculate severity ratings.	MobSF and the MARA Framework were used for automated static analysis and reverse engineering, together with a CWSS calculator for scoring.	Vulnerabilities were classified and rated using the OWASP Mobile Top 10 categories, CWE taxonomy IDs, and numerical CWSS scores.	Both apps have serious flaws across numerous OWASP categories, most notably using dangerous cryptography (SHA-1) and storing hardcoded secrets.	The investigation was limited to static analysis rather than dynamic runtime testing, which resulted in several false negatives during permission scanning.	Combines dual-tool verification with quantitative scoring to provide developers with meaningful risk prioritization for vital, government-backed health apps.
32	Cloud Computing-Based API Design and Implementation for Hening Mobile Application	2022	Hubert Igor Haryatmo Tandri; Hilal H. Nuha; and Rio G; Utomo	To overcome mobile resource limits and high backend costs, create a Cloud-based API (PaaS) for "Hening," a sign language translation app.	To facilitate communication, the program processes Bisindo (Indonesian Sign Language).	Deploys the application backend on Google App Engine (PaaS) to move heavy processing and storage responsibilities to the cloud.	Uses an Express.js-based API with Cloud SQL to convert sign language into text using conventional HTTP protocols.	Developed with Google Cloud Platform (App Engine, Cloud SQL), Express.js, and Kotlin, and tested with Postman.	Google Cloud Monitoring was used to assess system efficiency by measuring response latency, sent bytes, and CPU use.	Performance was incredibly efficient, with a peak response latency of only 6.07 ms and a CPU utilization of no more than 2%.	The Cloud SQL connector limits data retrieval to 150,000 rows and column headers to ASCII characters.	Provides a practical methodology for showing how PaaS may overcome mobile hardware limitations for resource-intensive, real-time translation tools.
33	Android Malware Detection and Its Security	2023	Pradeep Kumar Kushwaha; Pankaj Singh; Vikas Kumar; Rajesh Kumar; Manish Saraswat; and Swati Vashisht	To effectively identify Android malware using machine learning, we can overcome the limits of signature-based detection, which is easily avoided.	A dataset containing both benign and malicious Android applications.	We extracted static and dynamic features (permissions, API calls, and intentions) to test 10 different machine learning methods.	Extracted app methods, divided them into "strands," and compared them to the dataset with a Strand Similarity Measure.	The WEKA machine learning environment is based on methods such as Multilayer Perceptron (MLP) and Random Forest.	Accuracy, False Positives, Precision, Recall, F-measure, ROC, and RMSE were calculated using 10-fold cross-validation with a 66% split.	The MLP algorithm fared the best, with 99.40% accuracy and 0.60% false positive rate.	The dataset only had four attribute categories, necessitating additional testing on several datasets to rule out bias.	Proves that advanced models, such as MLP, outperform classic signature approaches and standard ML models utilized by anti-malware companies.
34	Review of Security Challenges in Mobile Cloud Computing Applications	2022	Majd Alqarqaz and Maram Bani Younes	Review security problems, threat models, and protection techniques for Mobile Cloud Computing	A thorough literature analysis that examines present theoretical MCC architectures and security studies.	MCC applications were categorized, active/passive threat models were created, and security	We investigated how MCC offloads significant processing and storage workloads from mobile devices to cloud servers.	Biometrics, encryption, firewalls, and intrusion detection systems are all components of the Secure Mobile Cloud (SMC) platform.	Qualitative evaluation of security procedures against predefined threat models.	Securing MCC necessitates an integrated, shared responsibility paradigm for cloud infrastructure and mobile OS systems.	No actual tests or unique quantitative techniques are used; instead, only secondary literature is consulted.	Emphasizes the necessity for synchronized security protocols that combine mobile OS security capabilities with cloud app store monitoring.

				(MCC) applications.		solutions were mapped across the physical, logical, and network layers.						
35	Android Malware Analysis Using Coefficient of Multiple Correlation	2023	Shresth Jain; Sarthak Kapoor; Sachin Kumar Sharma; and Anshul Arora	To improve Android malware detection, we use the Coefficient of Multiple Correlation (CMC) to rank and pick the most relevant static permissions, eliminating feature repetition.	A collection of 111,414 Android APKs, including 55,609 benign programs (confirmed by VirusTotal) and 55,805 malicious apps.	Permissions were extracted from the manifest file and ranked using CMC before being trained with several machine learning classifiers.	We decompiled APKs, calculated CMC scores for permission pairs, prioritized the features, and tested them using k-fold cross-validation.	Apktool, VirusTotal, and many ML/DL techniques such as Random Forest, XGBoost, and RNN.	The models were evaluated based on Classification Accuracy (%).	The Random Forest Classifier achieved the highest accuracy (97.54%) using the top 210 CMC-ranked permissions.	Static permission analysis is ineffective in detecting stealthy malware that seeks few permissions at first and then downloads harmful payloads.	Shows that applying a scalar CMC ranking successfully minimizes feature redundancy and improves machine learning detection models.
36	SafeDroid: Safeguarding Android Mobile Phones from Adware and Banking Maldroid Attacks	2023	Ankita Kumari and Ishu Sharma	To protect Android devices from adware and financial banking malware attacks.	Sourced from the Canadian Institute for Cybersecurity repository.	An AI-trained hardware chip (MLSTC) for real-time packet inspection has been proposed and implemented into the phone motherboard.	Intercepts incoming network data packets, preventing harmful information and allowing safe data thru.	The Machine Learning Smart Trained Chip (MLSTC) was tested using KNN, Random Forest, and CNN algorithms.	Accuracy, precision, recall, and the F1-score were used to assess performance.	Random Forest has the best categorization performance across all measures tested.	Proposes a physical motherboard chip solution but evaluates it solely on software-based simulation metrics, without validating hardware feasibility.	Moves threat defense away from typical software-level antivirus programs and toward an integrated hardware-software AI inspection solution.
37	Security Implications of Mobile Development Frameworks: Findings from Static Analysis of Android Apps	2024	Taras Fedynshyn, Olha Mykhaylova, and Ivan Opirskyy	To assess the security risks and privacy vulnerabilities of Android apps using various native and cross-platform frameworks.	6,165 top Android APK files selected based on Similarweb rankings.	Static analysis was performed using MobSF and linked to a Postgres database for post-processing.	High-severity concerns were identified by analyzing APK binaries, certificates, source code, hardcoded files, manifests, and permissions.	MobSF framework used to evaluate Android Native, React Native, Xamarin, Cordova, and Flutter.	The percentage of APKs with severe flaws and the average number of vulnerabilities per framework.	Frameworks revealed distinct flaws (e.g., Xamarin had high binary issues, React Native had the most code risks and dangerous permissions).	We relied only on automated static analysis, which may indicate genuine app permissions as false positives.	Shows that cross-platform frameworks create new, framework-specific vulnerabilities as compared to native programming.
38	Evaluating Security Features in Mobile Health Apps: A Systematic Review	2025	Yuanyuan Cao; Yi Xu and Leming Zhou	To conduct a systematic review of security implementations (confidentiality, integrity, and availability) in mHealth apps across various data sensitivity levels.	213 peer-reviewed publications were selected from 7,907 records in PubMed, IEEE Xplore, and CINAHL.	A PRISMA-guided systematic evaluation classified apps into three sensitivity tiers and compared security methods utilizing binary coding to the CIA triad.	We screened articles, collected app attributes, and compared security compliance trends to data sensitivity categories.	PubMed, IEEE Xplore, CINAHL databases, EndNote reference management, and the CIA Triad framework.	Specific confidentiality, integrity, and availability precautions are binary coded (present or absent).	Protections vary according to data sensitivity 75.9% of sensitive data apps have incorporated confidentiality restrictions, while integrity (37.4%) and availability (62.7%) protections are uneven across all categories.	Relied solely on self-reported details in published literature rather than actual technical testing; confined to English-language, open-access publications from three databases.	Demonstrates that while baseline confidentiality exists in mHealth, comprehensive safeguards for data integrity, availability, and explicit regulatory compliance (HIPAA/GDPR) are still severely lacking.
39	Securix: Mobile Security SDK	2024	Sonali Kothari; Tejas Ulbhaje; Vishant Kalwani; Yash Mangale and Pooja Bagane	To meet Reserve Bank of India (RBI) regulatory regulations, we developed a Kotlin-based security SDK ("Securix") that provides data obfuscation, anti-screenshot protection, and root detection for mobile banking applications.	Tested using a custom Kotlin demonstration banking application running on Android emulators and physical devices	The modular SDK architecture integrates security features directly into application execution paths and is validated via functional verification and penetration testing.	Sensitive text is obfuscated (except for the final four digits), FLAG_SECURE is enforced to prevent screen capture/recording, and test-keys, root paths, and the su binary are scanned.	Kotlin, Android SDK (WindowManager, LayoutParams, FLAG_SECURE), and RootChecker.	Functional verification and qualitative penetration testing (root blocking consistency, obfuscation accuracy, and screenshot interception).	Consistently banned screenshots across several OS versions/tools, effectively concealed user data, and restricted application execution in rooted circumstances.	Standard root-detection checks may be sensitive to advanced root-hiding technologies, and the study does not include empirical performance/overhead benchmarks.	Provides an integrated, developer-ready security SDK that simplifies compliance with financial regulatory requirements (RBI directives) in mobile Android applications.

40	Securing Flutter Applications: A Comprehensive Study	2023	Palash Rambhia; Dr. Kalyan Bamane and Parth Shinde	To dispel the misconception that Flutter apps are inherently secure due to native compilation by evaluating their vulnerabilities and defining essential security best practices.	Analyzed existing literature, global mobile cyber-attack statistics (2022), and functional Flutter/Dart code samples.	Structured review and code-level demonstration of Flutter security measures at the authentication, network transport, and data storage layers.	Flutter code methods were demonstrated as secure versus insecure, with obligatory user authentication before to database operations and encrypted local key-value storage.	Flutter framework, Dart, Firebase Authentication and Firestore, Dio networking library, Flutter Secure Storage plugin, OWASP ZAP, and Burp Suite.	A qualitative assessment of mobile cyber protection measures against data breaches, man-in-the-middle attacks, and reverse engineering.	Demonstrates that native Flutter binaries are still vulnerable to dynamic modification unless secured by Zero Trust architecture, HTTPS/TLS interceptors, and encrypted storage.	Relies on qualitative reviews and conceptual code samples rather than automated vulnerability scans or empirical benchmark testing on production apps.	Addresses a critical developer knowledge gap by providing an actionable, framework-specific guide to securing cross-platform Flutter applications.
----	--	------	--	---	---	--	--	---	---	--	--	--

3.1.1 Research Gap Analysis

- a. Research Gap 1.** Limited integration of static and dynamic security testing. Many studies rely heavily on Static Application Security Testing (SAST), which is effective for analysing application code, permissions and configurations but still may fail to detect vulnerabilities that appear only during runtime. The reviewed studies also show differences in the coverage of individual SAST tools, while hybrid approaches combining static and dynamic analysis provide better detection of runtime threats. However, the integration of these techniques into standard development workflows remains limited.
- b. Research Gap 2.** Limited real world and diverse datasets. Several studies use small datasets, selected applications or controlled testing environments. Specific sector of studies also focuses on applications from limited geographic locations or specific categories. As a result, the findings may not fully represent the diversity of vulnerabilities found across the wider mobile application ecosystem.
- c. Research Gap 3.** Limited detection of evolving threats. Machine learning based approaches have achieved high detection accuracy. However, its performance is often evaluated using existing and controlled datasets. This creates uncertainty of its ability to identify previously unseen malware and new attack techniques. Static analysis may also fail to detect malicious behaviour that appears only after installation or during execution.
- d. Research Gap 4.** Security limitations in cross platform applications. The growing use of frameworks such as Flutter, React Native and Xamarin introduces framework specific attack vectors. The reviewed studies indicate that automated security tools may struggle to accurately analyse intermediate binaries and runtime behaviour in cross platform applications.